

Compiladores 2026–2

Práctica 1

Análisis léxico con flex

Prof. Ariel Adara Mercado Martínez

Ayud. Carlos Gerardo Acosta Hernández

Adud. Luis Mario Escobar Rosales

Rodrigo André Decuir Fuentes

No. Cuenta: 320022711

ojo@ciencias.unam.mx

Fecha de entrega: 10 de marzo



Pasos

- Transcribir el código anterior a un archivo con extensión .ll, .lex o .flex dentro de la carpeta *src/Primer programa en Lex/*
- Compilar mediante la instrucción: `flex++ archivo.ll`
- Comprobar que se genere el archivo *lex.yy.cc*
- Compilar mediante: `g++ lex.yy.cc -o nombreEjecutable`
- Ejecutar mediante: `./nombreEjecutable`

Ejercicios

- ¿Qué ocurre si en la primera sección se quitan las llaves al nombre de la macro letra? (0.5 pts) **Respuesta:** ya no se toma letra como la macro letra [a-zA-Z], sino como la cadena "letra", de modo que solo se reconoce la palabra "letra".
- ¿Qué ocurre si en la segunda sección se quitan las llaves a los macros? (0.5 pts) **Respuesta:** se va a reconocer la palabra "digito" como un número y la palabra "palabra" como una palabra. Cualquier otra cadena no será reconocida.
- ¿Cómo se escribe un comentario en flex? (0.5 pts) **Respuesta:** se escriben igual que los comentarios de varias líneas en C++: `/* esto es un comentario */`
- ¿Qué se guarda en yytext? (0.5 pts) **Respuesta:** el texto que el analizador léxico (yylex) ya encontró y separó y separó basándose en una regla.
- ¿Qué pasa al ejecutar el programa e introducir cadenas de caracteres y de dígitos por la consola? (0.5 pts) **Respuesta:** Las cadenas de caracteres son reconocidas como palabras y los dígitos son reconocidos como números.
- ¿Qué ocurre si introducimos caracteres como "*" en la consola? (0.5 pts) **Respuesta:** No son reconocidos, ni como palabras, ni como números.
- Modificar al código anterior en un archivo nuevo, de tal manera que reconozca lo siguiente: (2 pts)
 - La expresión regular para los hexadecimales en lenguaje C++.
 - 5 palabras reservadas del lenguaje C++.
 - Los identificadores válidos del lenguaje C++, con longitud máxima de 32 caracteres (**Sugerencia:** use el operador {m,n}).
 - Los espacios en blanco.

Respuesta: Archivo con el código modificado: `Primer_programa_en_Lex/archivo.ll`

- Describir el conjunto de terminales y la expresión regular que reconoce a cada uno en *lexer.ll*. (2 pts) **Listo! Archivo modificado:** `C.1/lexer.ll`
- Generar acciones léxicas para cada terminal de nuestro lenguaje en *Lexer.cpp*, de modo que se muestre en pantalla la salida esperada con el archivo *prueba*. (2 pts) **Listo! Archivo modificado:** `C.1/lexer.ll`

10. Crear un *Makefile*. (1 pt) **Listo! El Makefile se encuentra dentro de C_1. Se compila con -make all- y se corre el programa con el test Prueba mediante -make run-: C_1/lexer.ll**
11. Documentar el código. (0.25pts)
12. Proponer 4 archivos de prueba nuevos, 2 válidos y 2 inválidos. (0.25pts)